

Shiv Nadar University Chennai

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective:

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

3. Dataset

Dataset: Fashion-MNIST
Training Images: 60,000
Testing Images: 10,000
Classes: 10
Image Size: 28×28

4. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

5. Source Code

Github Link:<https://github.com/PranavVarshin/DeepLearning/tree/main>

6. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.

```
def create_model(neurons1=128,
                 neurons2=64,
                 learning_rate=0.001):

    model = Sequential()

    model.add(Dense(neurons1,
                    activation='relu',
                    input_shape=(784,)))

    model.add(Dense(neurons2,
                    activation='relu'))

    model.add(Dense(10,
                    activation='softmax'))

    model.compile(
        optimizer=Adam(learning_rate=learning_rate),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    return model
```

Figure 1: BaseLine MLP Model

2. Define the hyperparameter search space.

```
#HyperParameter Search Space
param_grid = {
    "neurons1": [64, 128],
    "neurons2": [32, 64],
    "learning_rate": [0.001, 0.01],
    "batch_size": [32, 64],
    "epochs": [5, 10]
}
```

Figure 2: Hyperparameter Search Space

3. Perform Grid Search or Randomized Search using 5-fold cross-validation.

```
> #5fold cross validation
kfold = KFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

best_accuracy = 0
best_params = None

results = []

for params in ParameterGrid(param_grid):

    fold_scores = []

    print("\nTesting:", params)

    for train_index, val_index in kfold.split(X_train_flat):

        X_train_fold = X_train_flat[train_index]
        X_val_fold = X_train_flat[val_index]

        y_train_fold = y_train_cat[train_index]
        y_val_fold = y_train_cat[val_index]

        model = create_model(
            neurons1=params["neurons1"],
            neurons2=params["neurons2"],
            learning_rate=params["learning_rate"]
        )

        model.fit(
            X_train_fold,
            y_train_fold,
            epochs=params["epochs"],
            batch_size=params["batch_size"],
            verbose=0
        )

        predictions = model.predict(
            X_val_fold,
            verbose=0
        )

        predictions = np.argmax(predictions, axis=1)
        actual = np.argmax(y_val_fold, axis=1)

        acc = accuracy_score(actual, predictions)

        fold_scores.append(acc)

    mean_accuracy = np.mean(fold_scores)

    results.append({
        "Parameters": params,
        "Accuracy": mean_accuracy
    })

    print("Mean Accuracy:", mean_accuracy)

    if mean_accuracy > best_accuracy:

        best_accuracy = mean_accuracy
        best_params = params
```

Figure 3: 5 Fold Cross Validation

4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

7. Plots

8. Discussion

Answer the following:

1. Which optimization method was used?
2. Which hyperparameters were selected?
3. Did optimization improve performance?
4. Which hyperparameter had the greatest impact?
5. Compare Grid Search and Randomized Search.
6. Recommend the best MLP configuration.

9. Conclusion

10. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.